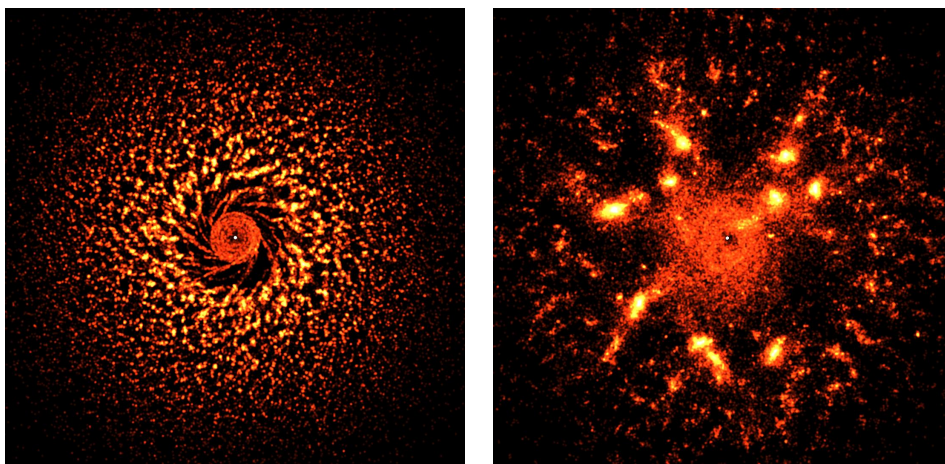


Паралелизация на N-Body симулация върху графичен процесор

Борис Василев
borisnv@uni-sofia.bg

Софийски Университет "Св. Климент Охридски"
Факултет по Математика и Информатика
10.06.2025



Абстракт

Проблемът N-Body е класически проблем в компютърната физика, който изисква изчисляване на гравитационните взаимодействия между N тела. Този доклад разглежда паралелизацията на този проблем върху графичен процесор (GPU) с цел ускоряване на изчисленията. Използван OpenGL за имплементация на алгоритъма, като се сравняват резултатите с последователно изпълнение върху CPU. Резултатите показват значително ускорение при използване на GPU, особено при големи стойности на N .

1 Анализ на проблема

Проблемът N-Body се състои в изчисляване на гравитационните сили между N тела, които могат да бъдат планети, звезди или други астрономически обекти. Всеки обект влияе на другите чрез гравитационна сила, която зависи от масата и разстоянието между тях. Целта е да се изчисли движението на телата във времето, което изисква многократни изчисления на взаимодействията.

Проблемът и методи за неговата паралелизация се разглеждат от самото създаване на машини, способни на паралелни изчисления [1]. Решения на проблема с N тела се използват във изчислителната астрофизика [2] и в оценката на графичен хардуер [3]. В литературата са се утвърдили няколко различни алгоритъма.

Изчисление по двойки (All-Pairs Method)

Най-простият метод за симулиране на гравитационните сили е пресмятането на ускоренията, породени от взаимодействията между всеки две тела. Алгоритъмът е разгледан в [4, 5, 6], а в [7] е разгледано негово разширение при разглеждане на негравитационни сили. Започвайки от закона за гравитацията на Нютон, можем да изразим ускорението на всяко тяло като сума от ускоренията, породени от всички останали тела. Нека като начало означим позициите на N -те тела с $\vec{r}_i$, масите им с m_i , разстоянието между телата i и j с $\vec{r}_{ij} = \vec{r}_i - \vec{r}_j$ и скоростите с $\vec{v}_i$. Тогава ускорението $\vec{a}_i$ може да се изрази като:

$$\begin{aligned}\vec{a}_i &= \frac{\sum_{j \neq i} \vec{F}_{ij}}{m_i} \\ &= \frac{\sum_{j \neq i} \frac{Gm_i m_j}{|\vec{r}_{ij}|^2} \frac{\vec{r}_{ij}}{|\vec{r}_{ij}|}}{m_i} \\ &= \sum_{j \neq i} \frac{Gm_j}{|\vec{r}_{ij}|^3} \vec{r}_{ij}\end{aligned}\tag{1}$$

Така ускорението е приведено във лесна за пресмятане форма като не изисква нормализиране на вектора $\vec{r}_{ij}$, а само изчисляване на неговата дължина. Извършването на пълната симулация след това може да се извърши чрез Ойлерово интегриране на ускорението. Означаваме данните на t -та стъпка със горен индекс (t) . Освен това, нека Δt е интеграционната стъпка. Забелязваме, че $\vec{F}_{ij} = \vec{F}_{ji}$ за всеки $i \neq j$. Така може да спестим пресмятане на ускоренията. Така псевдокода за цялостната симулация (за една стъпка) е описан в Алгоритъм 1.

Това е и алгоритъма, който се стремим да паралелизираме в този доклад. Той е лесен за разбиране и имплементиране, но има и недостатък:

Algorithm 1 N-Body симуляция по двойки обекти

```
1: for  $i \leftarrow 1$  to  $N$  do
2:    $\vec{v}_i^{(t+1)} \leftarrow \vec{v}_i^{(t)}$ 
3:   for  $j \leftarrow 1$  to  $i - 1$  do
4:      $\vec{diff} \leftarrow \vec{r}_j^{(t)} - \vec{r}_i^{(t)}$ 
5:      $\vec{F} \leftarrow G \cdot \frac{\vec{diff}}{|\vec{diff}|^3} \cdot \Delta t$ 
6:      $\vec{v}_i^{(t+1)} \leftarrow \vec{v}_i^{(t+1)} + m_j \cdot \vec{F}$ 
7:      $\vec{v}_j^{(t+1)} \leftarrow \vec{v}_j^{(t+1)} - m_i \cdot \vec{F}$ 
8:   end for
9: end for
10: for  $i \leftarrow 1$  to  $N$  do
11:    $\vec{r}_i^{(t+1)} \leftarrow \vec{r}_i^{(t)} + \vec{v}_i^{(t+1)} \cdot \Delta t$ 
12: end for
```

извършва $O(N^2)$ пресмятания, което го прави неефективен при голям брой тела.

Йерархични алгоритми

По-ефективни алгоритми за симуляция на N-Body проблемите използват йерархични структури от данни, които позволяват да се избегне изчисляването на взаимодействия между всички двойки тела. Най-често се използва структура, разделяща пространството рекурсивно до изпълняване на определен критерий (например клетката съдържа $\leq k$ тела). Взаимодействията в клетка-листо се извършват по метода на всички двойки, а между клетки се използват приближения на 'далечното векторно поле' (far-field approximation). Популярни алгоритми са Barnes-Hut [8] и Fast Multipole Method [9, 10]. Те са значително по-ефективни от метода на всички двойки, но също така са по-сложни за имплементиране и излизат от рамките на курсовата работа. В този доклад ще се фокусираме върху метода на всички двойки, тъй като той е най-прост и лесен за паралелизиране.

2 Реализация

Проучване

Реализация на N-Body симуляция по двойки обекти върху графичен процесор (GPU) е разглеждана в дълбочина от различни автори. [4, 5, 11] разглеждат основно CUDA само върху хардуер на NVIDIA, докато [6] поддържа и HIP върху AMD хардуер. Поради наличния хардуер, решихме да използваме OpenGL за имплементация на алгоритъма.

OpenGL е широко използван стандарт за графични изчисления и е съвместим с повечето графични карти, независимо от производителя. Това позволява по-широка съвместимост и лесно разпространение на приложението. По този начин постигаме добри ускорения дори и върху вградени графични процесори на Intel.

Паралелизация

Реализацията на алгоритъма до голяма степен се доближава до този в [5]. Основната разлика е, че използваме OpenGL Compute с GLSL (OpenGL Shading Language) за изпълнение на изчисленията върху GPU, вместо CUDA. В 1 можем да видим, че `for`-циклите `@1...@9` и `@11...@13` трябва да се извършат последователно, а всеки от тях може да бъде паралелизиран. Иначе казано, съставяме два етапа на изчисление: първият етап изчислява ускоренията и скоростите на телата, а вторият етап обновява позициите им. Всеки от тези етапи може да бъде изпълнен паралелно за всяко тяло. Поради високия хардуерен паралелизъм на графичните процесори, може за N тела да създадем N паралелно работещи нишки. При този подход, обаче, на редове `@6` и `@7` получаваме конкурентен достъп до скоростите на телата, което изисква допълнителна синхронизация между нишките. Всякаква глобална (между произволни две нишки) синхронизация би довела до голямо забавяне върху графичен процесор поради самата му архитектура. За да избегнем това, може да 'платим' като не се възползваме от факта, че $\vec{F}_{ij} = \vec{F}_{ji}$ и да изчисляваме ускоренията на телата напълно отделно. Накрая, за да избегнем числени грешки, породени от твърде близки разстояния между телата, не позволяваме изчисленото разстояние да е по-малко от предварително избрана константа ϵ . Така получаваме алгоритъмът 2. В [5] се прави и опит за подобряване на производителността за малки стойности на N , но тук се стремим да постигнем максимално ускорение при големи стойности на N и за това не разглеждаме тази оптимизация.

Имплементация

GPU имплементация

За реализиране на алгоритъма създаваме два OpenGL Compute Shader-a, които изпълняват двете стъпки в алгоритъм 2. Телата се съхраняват в буфер във традиционна 'Array of Structs' (AoS) форма. Този буфер се закача във вид на шейдърна споделена памет (Shader Storage Buffer Object - SSBO) за да може елементите да се достъпват от всички нишки. В алгоритъма не се налага различни нишки да достъпват същия елемент от буфера, така че не се налага синхронизация между нишките.

С оглед на архитектурата на графичния процесор, проблемът трябва да бъде разделен на блокове от нишки, представени в двумерна решетка.

Algorithm 2 Паралелна N-Body симулация по двойки обекти

```
1: for  $i \leftarrow 1$  to  $N$  in parallel do
2:    $\vec{v}_i^{(t+1)} \leftarrow \vec{v}_i^{(t)}$ 
3:   for  $j \leftarrow 1$  to  $N$  do
4:     if  $i = j$  then
5:       continue
6:     end if
7:      $\overrightarrow{diff} \leftarrow \vec{r}_j^{(t)} - \vec{r}_i^{(t)}$ 
8:      $d \leftarrow \max(|\overrightarrow{diff}|, \epsilon)$ 
9:      $\vec{F} \leftarrow G \cdot \frac{\overrightarrow{diff}}{d^3} \cdot \Delta t$ 
10:     $\vec{v}_i^{(t+1)} \leftarrow \vec{v}_i^{(t+1)} + m_j \cdot \vec{F}$ 
11:   end for
12: end for
13: for  $i \leftarrow 1$  to  $N$  in parallel do
14:    $\vec{r}_i^{(t+1)} \leftarrow \vec{r}_i^{(t)} + \vec{v}_i^{(t+1)} \cdot \Delta t$ 
15: end for
```

Поради споделения кеш между нишките в един Thread Block, е по-ефективно нишките в един блок да обработват тела, намиращи се в последователни клетки в паметта. Така при размер на блока (фиксиран за модел на графичния процесор) $p \times p$ и брой тела N , броят на блоковете е $\lceil N/p^2 \rceil$. При това разделение, всяка нишка изчислява ускорението на p тела, като използва данните от всички останали тела. Така цялото изчисление е разпределено в решетка от нишки $\lceil N/p^2 \rceil p \times p$. Ясно е, че при закръглянето на броя блокове, някои нишки ще останат без работа, но това е неизбежно.

CPU имплементация

За сравнение, имплементацията на CPU е направена с помощта на C++ и реализира последователния алгоритъм 1 при същата структура на данните. Имплементацията използва и SIMD където е възможно чрез библиотеката за векторно смятане GLM [12].

3 Тестове

Хардуер

Тестовите са проведени върху машина с процесор Intel Core i7-6700 и графичен процесор AMD Radeon RX 7600 (обозначен по-долу *GPU1*) с 2048 поточни процесора и операционна система EndeavourOS Linux 6.13.7. Втори тест е проведен върху машина с графичен процесор Radeon 530 Mobile GDDR5 (обозначен по-долу *GPU2*) с 384 поточни процесора.

Тестови данни

Тестовите данни са генерирани случайно и са нормално разпределени с дисперсия 30. За по-висока визуална атрактивност в точката $[0, 0]$ е поставено тяло, чиято маса е сумата на останалите тела. Това позволява да се наблюдава как останалите тела се движат около него. За целите на тестовете са използвани стойности на N от 10 до 10^5 , като за всяка стойност са извършени поне 10 симулации с различни начални условия (100 за $N \leq 1000$).

Резултати и анализ

На фигура 1 е показано сравнение на времето за изпълнение на симулацията върху CPU и GPU. Времето е измерено в микросекунди (μs) и е логаритмично скалирано спрямо броя на телата N . От графиката се вижда, че времето за изпълнение на симулацията върху GPU е значително по-ниско от това върху CPU, особено при големи стойности на N . На фигура 2 е показано ускорението на GPU спрямо CPU, което достига до над 400 пъти при $N = 10^5$. Това показва, че паралелизацията на N-Body симулацията върху GPU е изключително ефективна и може да се използва за значително ускоряване на изчисленията.

Остава да оценим оптималността на паралелизацията. Първо нека разгледаме *GPU1*. Знаем, че имплементацията на последователния алгоритъм спестява около половината операции чрез съображението по-горе. Освен това, всяко ядро на процесора е способно да работи на честота до 4 GHz, докато използвания графичния процесор не може да поддържа честоти над 2.3 GHz за дълго време. Така знаейки и максималния хардуерен паралелизъм на графичния ускорител - 2048 можем да изчислим грубо максималното теоретично ускорение, което може да се постигне. То е $S_{max} = 2048 \cdot \frac{2.3}{4} \cdot \frac{1}{2} \approx 588.8$.

Това е теоретичното максимално ускорение, което може да се постигне при идеални условия. Виждаме, че реалното ускорение достига до 425, и получаваме ефективност на паралелизацията $E = \frac{425}{588.8} \approx 0.722$, с което считаме, че паралелизацията е близко до оптимална.

В случая на *GPU2* можем да направим подобна оценка. С 384 поточни процесора и средна честота от 0.8 GHz, теоретичното максимално ускорение е $S_{max} = 384 \cdot \frac{0.8}{4} \cdot \frac{1}{2} \approx 38.4$ и достигнатото ускорение е $S = 24$, което дава ефективност $E = \frac{24}{38.4} \approx 0.625$. Това показва, че паралелизацията е по-малко ефективна върху по-слабия графичен процесор, но все пак постига значително ускорение спрямо CPU. Пълна таблица с резултати от тестовете е показана на фигура 4.

Figure 1: Графика на времето за изпълнение на симулацията

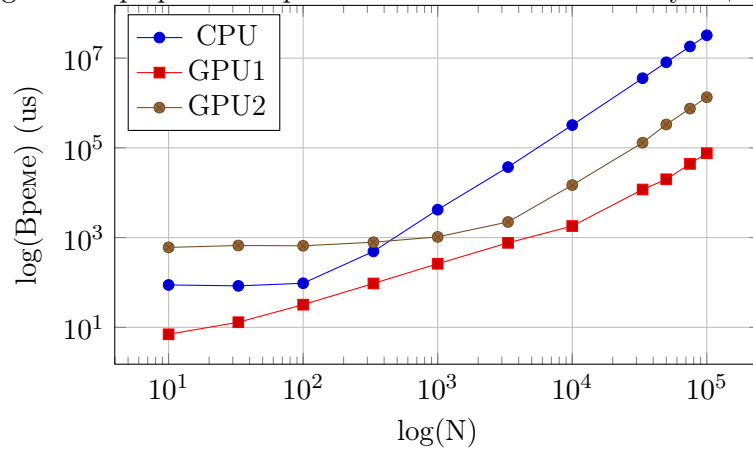


Figure 2: Графика ускорението на GPU спрямо CPU

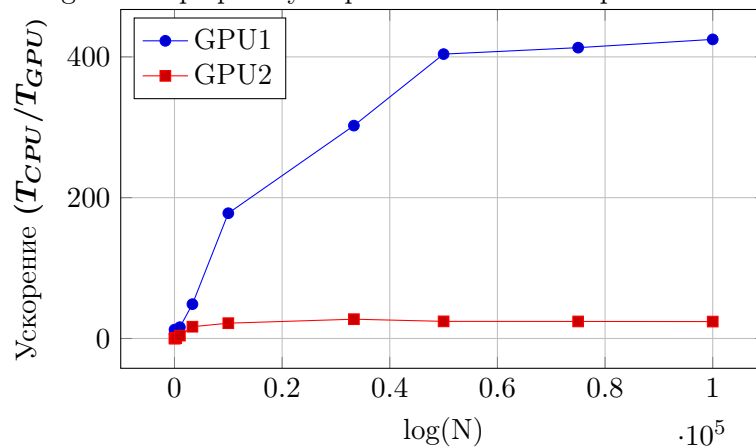


Figure 3: Графика на ефективността на паралелизацията

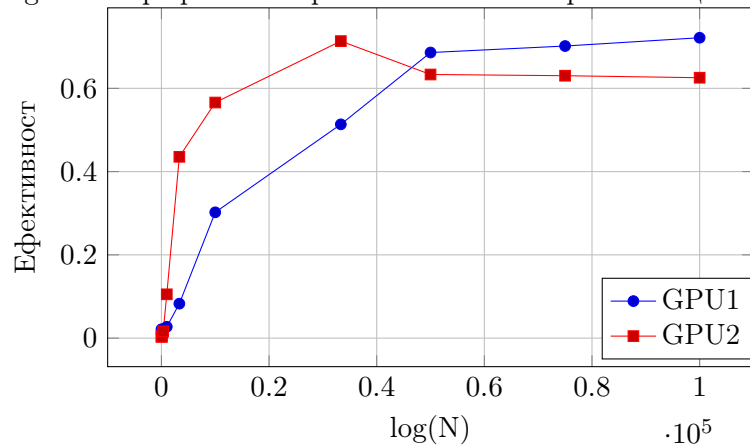


Figure 4: Таблица с резултати от тестовете

N	$T_{CPU}(\mu s)$	$T_{GPU1}(\mu s)$	S_1	E_1	$T_{GPU2}(\mu s)$	S_2	E_2
10	88	7	12.571	0.021	605	0.145	0.004
33	84	13	6.462	0.011	666	0.126	0.003
100	96	32	3.000	0.005	657	0.146	0.004
333	492	95	5.179	0.009	788	0.624	0.016
1000	4187	261	16.042	0.027	1036	4.042	0.105
3333	37246	764	48.751	0.083	2227	16.725	0.436
10000	321685	1808	177.923	0.302	14803	21.731	0.566
33333	3560951	11777	302.365	0.514	129982	27.396	0.713
50000	8054302	19940	403.927	0.686	331224	24.317	0.633
75000	18142061	43916	413.108	0.702	749560	24.204	0.630
100000	32220759	75842	424.841	0.722	1341493	24.019	0.625

Тук T_{CPU} , T_{GPU1} и T_{GPU2} са времето за изпълнение на симулацията върху CPU , $GPU1$ и $GPU2$ съответно, S_1 и S_2 са ускорението на $GPU1$ и $GPU2$ спрямо CPU , E_1 и E_2 са ефективността на паралелизацията върху $GPU1$ и $GPU2$, пресметнати спрямо съображенията по-горе.

References

- [1] S. J. Aarseth, S. J. Aarseth, Gravitational N-body simulations: tools and algorithms, Cambridge University Press, 2003.
- [2] C. Lacey, S. Cole, Merger rates in hierarchical models of galaxy formation—ii. comparison with n-body simulations, *Monthly Notices of the Royal Astronomical Society* 271 (3) (1994) 676–692.
- [3] D. P. Playne, M. Johnson, K. A. Hawick, Benchmarking gpu devices with n-body simulations., *CDES* 9 (2009) 132.
- [4] M. Burtscher, K. Pingali, An efficient cuda implementation of the tree-based barnes hut n-body algorithm, in: *GPU computing Gems Emerald edition*, Elsevier, 2011, pp. 75–92.
- [5] L. Nylons, M. Harris, J. Prins, Fast n-body simulation with cuda, *GPU gems* 3 (2007) 62–66.
- [6] S. L. Grimm, J. G. Stadel, R. Brasser, M. M. M. Meier, C. Mordasini, Genga. ii. gpu planetary n-body simulations with non-newtonian forces and high number of particles, *The Astrophysical Journal* 932 (2) (2022) 124. doi:10.3847/1538-4357/ac6dd2.
URL <http://dx.doi.org/10.3847/1538-4357/ac6dd2>
- [7] R. Spurzem, Direct n-body simulations, *Journal of Computational and Applied Mathematics* 109 (1-2) (1999) 407–432.
- [8] J. Barnes, P. Hut, A hierarchical o ($n \log n$) force-calculation algorithm, *nature* 324 (6096) (1986) 446–449.
- [9] L. Greengard, V. Rokhlin, A fast algorithm for particle simulations, *Journal of computational physics* 73 (2) (1987) 325–348.
- [10] W. Dehnen, A fast multipole method for stellar dynamics, *Computational Astrophysics and Cosmology* 1 (2014) 1–23.
- [11] R. G. Belleman, J. Bédorf, S. F. P. Zwart, High performance direct gravitational n-body simulations on graphics processing units ii: An implementation in cuda, *New Astronomy* 13 (2) (2008) 103–112.
- [12] C. Riccio, *Glm: Manual* (2010).